Helpdesk mock API

Server finto per la prova pratica. I dati stanno in memoria: a ogni avvio si riparte dai dati iniziali.

Avvio

```
npm install
npm start
```

Il server ascolta su `http://localhost:3000`. CORS è aperto, quindi puoi chiamarlo direttamente da `http://localhost:4200`.

La documentazione interattiva (Swagger UI) è su `http://localhost:3000/docs`: fai il login da lì, copia il token nel pulsante **Authorize** e prova gli endpoint. La specifica OpenAPI è su `http://localhost:3000/openapi.json`.

Regole generali

- Ogni risposta arriva con una latenza casuale di 200-800 ms.
- Tutte le route `/api/*` tranne il login richiedono `Authorization: Bearer <token>`. Senza token o con un token non valido la risposta è `401`.
- Gli errori hanno sempre questa forma. `errors` compare solo sui `400`:

```
{
  "message": "Validation failed",
  "errors": { "title": ["Title must be between 5 and 100 characters"] }
}
```

- Le route con `:id` rispondono `404` se il ticket non esiste, compresi stato e commenti.
- `status`: `OPEN` | `IN_PROGRESS` | `RESOLVED` | `CLOSED`
- `priority`: `LOW` | `MEDIUM` | `HIGH` | `URGENT`

Utente di prova: `mario.rossi` / `password`.

Endpoint

POST /api/auth/login

Body: `{ "username": "mario.rossi", "password": "password" }` Risposta `200`: `{ "token": "...", "user": { "id": 1, "username": "mario.rossi", "fullName": "Mario Rossi", "role": "AGENT" } }` Credenziali errate: `401`.

GET /api/me

Utente associato al token.

GET /api/users

Elenco utenti (per assegnatario e filtri): `[{ "id": 1, "username": "mario.rossi", "fullName": "Mario Rossi", "role": "AGENT" }, ...]`. `role` è `AGENT` o `REQUESTER`. Come assegnatario di un ticket il server accetta qualunque utente esistente, `REQUESTER` compresi.

GET /api/categories

Elenco categorie: `[{ "id": 1, "name": "Hardware" }, ...]`.

GET /api/tickets

Query string, tutti i parametri sono opzionali:

Parametro	Esempio	Note
`page`	`1`	parte da 1
`pageSize`	`10`	massimo 100
`status`	`OPEN`	
`priority`	`HIGH`	
`categoryId`	`3`	
`assigneeId`	`2`	
`search`	`vpn`	cerca in titolo e descrizione
`sort`	`createdAt,desc`	campi: `createdAt`, `updatedAt`, `title`, `priority`; direzione `asc` o `desc`

Risposta `200`:

```
{
  "items": [
    {
      "id": 7,
      "title": "Printer offline (#7)",
      "description": "...",
      "status": "OPEN",
      "priority": "HIGH",
      "categoryId": 1,
      "assigneeId": 2,
      "reporterId": 4,
      "createdAt": "2026-09-14T10:00:00.000Z",
      "updatedAt": "2026-09-15T08:30:00.000Z"
    }
  ],
  "total": 60,
  "page": 1,
```

```
  "pageSize": 10
}
```

`assigneeId` può essere `null`.

GET /api/tickets/:id

Dettaglio: stesso oggetto degli `items` della lista. `404` se non esiste.

POST /api/tickets

Body: `{ "title", "description", "priority", "categoryId", "assigneeId"? }`

- `title`: da 5 a 100 caratteri
- `description`: obbligatoria
- `priority`: uno dei valori ammessi
- `categoryId`: deve esistere
- `assigneeId`: facoltativo, se presente deve esistere

Risposta `201` con il ticket creato (stato `OPEN`). Dati non validi: `400` con `errors` per campo.

PUT /api/tickets/:id

Stesso body e stessa validazione del `POST`. `404` se il ticket non esiste, `409` se è `CLOSED`.

PATCH /api/tickets/:id/status

Body: `{ "status": "IN_PROGRESS" }` Passaggi ammessi: `OPEN → IN_PROGRESS`, `IN_PROGRESS → RESOLVED`, `RESOLVED → IN_PROGRESS`, `RESOLVED → CLOSED`. Un passaggio non ammesso dà `422` con il motivo in `message`. Uno stato sconosciuto dà `400`.

GET /api/tickets/:id/comments

Commenti del ticket, dal più vecchio al più recente: `[{ "id", "ticketId", "authorId", "authorName", "text", "createdAt" }]`.

POST /api/tickets/:id/comments

Body: `{ "text": "..." }`. L'autore è l'utente del token. Risposta `201`. Testo vuoto: `400`.

GET /api/stats

`{ "total": 60, "byStatus": { "OPEN": 15, ... }, "byPriority": { "LOW": 14, ... } }` Questo endpoint risponde `500` in circa un quarto delle chiamate. È voluto.